

Data Abstraction, Cleaning & Processing — Data Science & ML Fundamentals

Data cleaning and preprocessing is the most heavily tested **practical** skill across MAANG (Meta, Amazon, Apple, Netflix, Google) Data Scientist, Data Analyst, and ML Engineer interviews — it shows up in SQL screens, Python/pandas coding rounds, take-home case studies, and ML system design. This guide covers data abstraction fundamentals through production-grade cleaning and processing, with definitions, formulas, runnable code, and interview framing for each topic.

Contents

WHAT THIS GUIDE COVERS

- 1. Why This Matters for MAANG Interviews

- 2. Data Abstraction Fundamentals

- 3. Data Types & Structures

- 4. Data Ingestion

- 5. Data Cleaning

- 6. Data Transformation & Feature Engineering

- 7. Handling Imbalanced Data

- 8. Data Aggregation & Reshaping

- 9. Data Validation & Pipelines

- 10. SQL for Data Cleaning

- 11. Big Data & Scaling

- 12. Common MAANG Interview Questions

- 13. Data Cleaning Checklist

- 14. Suggested Prep Plan

01

Why This Matters for MAANG Interviews

- Practitioners spend an estimated 60–80% of real-world project time on data cleaning and preparation, not modeling — interviewers know this and test for it directly.
- Where it shows up:
- **SQL screens** — deduplication, NULL handling, window functions
- **Python/pandas coding rounds** — given a messy CSV, clean and analyze it live
- **Take-home case studies** — full EDA + cleaning + feature engineering + modeling
- **ML system design** — "design a pipeline that ingests, validates, and cleans data for a feature store"
- **Behavioral rounds** — "tell me about a time you found and fixed bad data"

02 Data Abstraction Fundamentals

• What Is Data Abstraction?

Definition: Representing data at different levels of detail — hiding low-level storage/implementation details behind higher-level, task-relevant representations.

Purpose: reason about and manipulate data without worrying about how it's physically stored

Use Cases: data model design, feature pipelines, database schema design

The Abstraction Ladder:

FORMULA

Raw Data → Cleaned Data → Structured/Tidy Data → Features → Model Input → Prediction

• Structured vs Semi-Structured vs Unstructured Data

Type	Description	Examples	Typical Storage
Structured	Fixed schema, rows/columns	SQL tables, CSVs	RDBMS (MySQL, Postgres)
Semi-structured	Flexible, self-describing schema	JSON, XML, logs	Document stores (MongoDB), S3
Unstructured	No predefined schema	images, text, audio, video	data lakes, blob storage

- **Data Measurement Scales**

Scale	Description	Example
Nominal	categories, no order	color, country
Ordinal	categories, ordered	rating (low/med/high)
Interval	numeric, no true zero	temperature (°C)
Ratio	numeric, true zero	age, income, height

 **Interview Tip:** "Is this variable categorical or continuous, and how would you encode it?" is a very common opener — the measurement scale tells you the right encoding and statistic to use.

03 Data Types & Structures

- **Scalars, Vectors, Matrices, Tensors**

Definition: Core numeric abstractions used throughout ML — a scalar is a single number, a vector is 1D, a matrix is 2D, a tensor is N-dimensional.

Code:

PYTHON

```
import numpy as np

scalar = 5
vector = np.array([1, 2, 3])           # shape (3,)
matrix = np.array([[1, 2], [3, 4]])    # shape (2,2)
tensor = np.random.rand(3, 4, 5)      # shape (3,4,5)
```

- **Series vs DataFrame (pandas)**

Definition: A `Series` is a 1D labeled array; a `DataFrame` is a 2D labeled table of Series sharing an index.

Code:

PYTHON

```
import pandas as pd

s = pd.Series([1, 2, 3], name="age")
df = pd.DataFrame({"age": [25, 30], "city": ["NYC", "SF"]})
```

• Python Data Structures — Interview Complexity Cheat Sheet

Structure	Lookup	Insert	Use When
<code>list</code>	$O(n)$	$O(1)$ amortized append	ordered, allows duplicates
<code>dict</code>	$O(1)$ avg	$O(1)$ avg	key-value lookups
<code>set</code>	$O(1)$ avg	$O(1)$ avg	uniqueness, membership tests
<code>tuple</code>	$O(n)$	immutable	fixed, hashable records

 **Interview Tip:** Swapping a `list` for a `set` / `dict` to speed up membership checks is a common "optimize this" follow-up in coding rounds.

04 Data Ingestion

Definition: Reading raw data from its source format into a working representation.

Purpose: get external data into a form you can clean and analyze

Code:

PYTHON

```
import pandas as pd

df_csv      = pd.read_csv("data.csv")
df_json     = pd.read_json("data.json")
df_parquet  = pd.read_parquet("data.parquet")

# SQL
import sqlalchemy
engine = sqlalchemy.create_engine("postgresql://user:pass@host/db")
df_sql = pd.read_sql("SELECT * FROM users", engine)

# REST API
import requests
data = requests.get("https://api.example.com/data").json()
df_api = pd.DataFrame(data)
```


05 Data Cleaning

The most heavily interviewed practical skill: detecting and fixing missing values, duplicates, outliers, and malformed data.

- **5.1 Missing Value Detection**

Code:

PYTHON

```
df.isnull().sum()           # count missing per column
df.info()                   # dtype + non-null counts
df.isnull().mean() * 100    # % missing per column
```

• 5.2 Missing Value Imputation

Strategy	When to Use
Drop rows/columns	missingness is small (<5%) and random
Mean/Median imputation	numeric, symmetric/skewed distributions
Mode imputation	categorical features
Forward/backward fill	time series data
KNN imputation	missingness correlated with other features
Model-based (iterative)	complex, multivariate missingness patterns

Formula:

FORMULA

Mean = $(1/n) \sum x_i$

Median = middle value of sorted x_i (robust to outliers)

Code:

PYTHON

```
df.dropna(subset=["col"], inplace=True)

df["col"].fillna(df["col"].mean(), inplace=True)
df["col"].fillna(df["col"].median(), inplace=True)
df["col"].fillna(df["col"].mode()[0], inplace=True)

df["col"].fillna(method="ffill", inplace=True) # forward fill
df["col"].fillna(method="bfill", inplace=True) # backward fill

from sklearn.impute import KNNImputer
imputer = KNNImputer(n_neighbors=5)
X_imputed = imputer.fit_transform(X)
```

 **Interview Tip:** Always ask *why* data is missing (MCAR, MAR, MNAR) before picking a strategy — interviewers reward that reasoning as much as the code.

• 5.3 Duplicate Detection & Removal

Code:

PYTHON

```
df.duplicated().sum()
df.drop_duplicates(subset=["user_id"], keep="first", inplace=True)
```

• 5.4 Outlier Detection

Formula (Z-score method):

FORMULA

$$z = (x - \mu) / \sigma \quad \rightarrow \text{flag if } |z| > 3$$

Formula (IQR method):

FORMULA

Q1 = 25th percentile, Q3 = 75th percentile, IQR = Q3 - Q1
Outlier if $x < Q1 - 1.5 \cdot \text{IQR}$ or $x > Q3 + 1.5 \cdot \text{IQR}$

Code:

PYTHON

```
# Z-score
from scipy import stats
df["z_score"] = stats.zscore(df["value"])
outliers = df[df["z_score"].abs() > 3]

# IQR
Q1, Q3 = df["value"].quantile([0.25, 0.75])
IQR = Q3 - Q1
lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
outliers = df[(df["value"] < lower) | (df["value"] > upper)]

# Isolation Forest (multivariate outliers)
from sklearn.ensemble import IsolationForest
iso = IsolationForest(contamination=0.05)
df["is_outlier"] = iso.fit_predict(df[["value1", "value2"]])
```

• 5.5 Data Type Conversion & Parsing

Code:

PYTHON

```
df["date"]      = pd.to_datetime(df["date"], errors="coerce")
df["price"]     = pd.to_numeric(df["price"], errors="coerce")
df["category"]  = df["category"].astype("category")
```

• 5.6 String Cleaning & Normalization

Code:

PYTHON

```
df["name"]  = df["name"].str.strip().str.lower()
df["phone"] = df["phone"].str.replace(r"\D", "", regex=True)

# Fuzzy matching for inconsistent categories ("NYC" vs "New York City")
from rapidfuzz import process
match, score, _ = process.extractOne("NYC", ["New York City", "Los Angeles"])
```

• 5.7 Data Validation

Definition: Enforcing schema and constraint rules so bad data is caught before it reaches modeling.

Purpose: fail fast on malformed data rather than silently corrupting downstream results

Code:

PYTHON

```
import pandera as pa

schema = pa.DataFrameSchema({
    "age": pa.Column(int, pa.Check.in_range(0, 120)),
    "email": pa.Column(str, pa.Check.str_matches(r"^[^@]+@[^@]+\.[^@]+$")),
})
schema.validate(df)
```

- 6.1 Feature Scaling

Formula:

FORMULA

Min-Max: $x' = (x - \min) / (\max - \min)$ → range [0,1]
Standardization: $x' = (x - \mu) / \sigma$ → mean 0, std 1
Robust Scaling: $x' = (x - \text{median}) / \text{IQR}$ → resistant to outliers

Code:

PYTHON

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler

X_scaled = MinMaxScaler().fit_transform(X)
X_scaled = StandardScaler().fit_transform(X)
X_scaled = RobustScaler().fit_transform(X)
```

• 6.2 Encoding Categorical Variables

Method	Best For
One-Hot Encoding	low-cardinality nominal features
Label Encoding	ordinal features (natural order)
Target/Mean Encoding	high-cardinality features, tree models
Frequency Encoding	high-cardinality, preserves distribution info

Formula (Target Encoding):

FORMULA

$\text{encoded}(\text{category}) = \text{mean}(\text{target} \mid \text{category})$

Code:

PYTHON

```
pd.get_dummies(df, columns=["city"]) # one-hot

from sklearn.preprocessing import LabelEncoder
df["size_enc"] = LabelEncoder().fit_transform(df["size"])

import category_encoders as ce
encoder = ce.TargetEncoder(cols=["city"])
df["city_enc"] = encoder.fit_transform(df["city"], df["target"])

df["city_freq"] = df["city"].map(df["city"].value_counts()) # frequency
```

• 6.3 Feature Creation

Code:

PYTHON

```
# Binning
df["age_bucket"] = pd.cut(df["age"], bins=[0, 18, 35, 60, 100],
                           labels=["teen", "young_adult", "adult", "senior"])

# Date/time features
df["year"] = df["date"].dt.year
df["month"] = df["date"].dt.month
df["dow"] = df["date"].dt.dayofweek

# Cyclical encoding for periodic features (e.g. hour of day)
df["hour_sin"] = np.sin(2 * np.pi * df["hour"] / 24)
df["hour_cos"] = np.cos(2 * np.pi * df["hour"] / 24)

# Interaction / polynomial features
from sklearn.preprocessing import PolynomialFeatures
X_poly = PolynomialFeatures(degree=2).fit_transform(X)
```

• 6.4 Text Preprocessing

Formula (TF-IDF):

FORMULA

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \log(N / \text{DF}(t))$$

Code:

PYTHON

```
import re
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

text = re.sub(r"[^a-zA-Z\s]", "", text.lower())
tokens = [w for w in text.split() if w not in stopwords.words("english")]
tokens = [WordNetLemmatizer().lemmatize(w) for w in tokens]

from sklearn.feature_extraction.text import TfidfVectorizer
X_tfidf = TfidfVectorizer(max_features=5000).fit_transform(corpus)
```

07

Handling Imbalanced Data

Definition: Techniques for classification datasets where one class vastly outnumbers another.

Purpose: prevent models from being biased toward the majority class

Use Cases: fraud detection, churn prediction, rare disease detection

Formula (SMOTE — Synthetic Minority Oversampling):

FORMULA

$$x_{\text{new}} = x_i + \lambda \cdot (x_{\text{neighbor}} - x_i) \quad \lambda \in [0,1], x_{\text{neighbor}} = \text{a } k\text{-NN of } x_i$$

Code:

PYTHON

```
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler

X_res, y_res = SMOTE().fit_resample(X, y)
X_res, y_res = RandomUnderSampler().fit_resample(X, y)

# Or use class weighting instead of resampling
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(class_weight="balanced")
```


08 Data Aggregation & Reshaping

- **GroupBy & Pivoting**

Code:

PYTHON

```
df.groupby("category")["sales"].agg(["mean", "sum", "count"])
df.pivot_table(index="region", columns="year", values="revenue", aggfunc="sum")
df.melt(id_vars=["id"], value_vars=["jan", "feb"], var_name="month", value_name="sale")
```

- **Merging & Joining**

Join Type	Keeps
Inner	rows matching in both tables
Left	all left rows + matches from right
Right	all right rows + matches from left
Outer	all rows from both, NaN where unmatched

Code:

PYTHON

```
pd.merge(df1, df2, on="user_id", how="inner")
pd.merge(df1, df2, on="user_id", how="left")
```

- **Window Functions**

Code:

PYTHON

```
df["rolling_avg"] = df["sales"].rolling(window=7).mean()
df["rank"] = df.groupby("category")["sales"].rank(method="dense", ascending=False)
```

Definition: ETL (Extract-Transform-Load) transforms data before loading it into storage; ELT loads raw data first and transforms it inside the warehouse.

Purpose: build repeatable, testable, production-grade data flows

Code (scikit-learn Pipeline):

PYTHON

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_pipeline, ["age", "income"]),
    ("cat", categorical_pipeline, ["city", "gender"])
])

X_processed = preprocessor.fit_transform(df)
```

10 SQL for Data Cleaning

MAANG SQL rounds heavily test these exact patterns.

NULL Handling:

SQL

```
SELECT COALESCE(email, 'unknown') AS email FROM users;
SELECT * FROM users WHERE email IS NULL;
```

Deduplication (keep most recent row per user):

SQL

```
WITH ranked AS (
  SELECT *, ROW_NUMBER() OVER (
    PARTITION BY user_id ORDER BY created_at DESC
  ) AS rn
  FROM events
)
SELECT * FROM ranked WHERE rn = 1;
```

Outlier detection with percentiles:

SQL

```
SELECT *
FROM orders
WHERE amount > (SELECT PERCENTILE_CONT(0.99) WITHIN GROUP (ORDER BY amount) FROM orders)
```

String cleaning:

SQL

```
SELECT TRIM(LOWER(REGEXP_REPLACE(name, '[^a-zA-Z ]', ''))) AS clean_name FROM customers
```

 **Interview Tip:** `ROW_NUMBER() OVER (PARTITION BY ...)` for deduplication is one of the most frequently asked SQL patterns in FAANG onsite loops.

Definition: Techniques for cleaning/processing data too large to fit in memory.

Code:

PYTHON

```
# Chunked processing
for chunk in pd.read_csv("huge.csv", chunksize=100_000):
    process(chunk)

# Dask (pandas-like, out-of-core)
import dask.dataframe as dd
ddf = dd.read_csv("huge.csv")

# PySpark
from pyspark.sql import SparkSession
spark = SparkSession.builder.getOrCreate()
df = spark.read.csv("huge.csv", header=True, inferSchema=True)
df = df.dropDuplicates().na.drop()

# Memory optimization
df["category"] = df["category"].astype("category")
df["small_int"] = pd.to_numeric(df["small_int"], downcast="integer")
```

12

Common MAANG Interview Questions

SQL - Remove duplicate rows, keeping only the latest record per user. - Find and appropriately handle NULL values in a sales table. - Identify outlier transactions using percentiles or window functions.

Python / Pandas - Given a messy CSV, clean it and report % missing per column. - Detect and impute missing values appropriately for a mixed numeric/categorical dataset. - Merge two datasets with mismatched keys or inconsistent formats.

Case Study / Take-Home - "Here's raw event log data — clean it, engineer features, and build a baseline model." - "How would you detect and handle data drift in a production pipeline?"

System Design - "Design a data pipeline that ingests, validates, and cleans data before it reaches a feature store."

13 Data Cleaning Checklist

- ☐ Inspect shape, dtypes, and first/last rows
- ☐ Quantify missing values per column
- ☐ Check for duplicates
- ☐ Validate data types match expectations
- ☐ Detect outliers (univariate + multivariate)
- ☐ Standardize categorical labels (casing, typos)
- ☐ Check class balance for classification targets
- ☐ Verify joins don't create unintended duplicates
- ☐ Document every cleaning decision and its justification

14

Suggested Prep Plan

1. Python fundamentals + NumPy/Pandas basics
2. SQL joins, aggregations, window functions (LeetCode SQL, StrataScratch)
3. Missing value & outlier handling techniques
4. Encoding & scaling methods
5. Imbalanced data techniques
6. Practice on messy real-world datasets (Kaggle)
7. Build one end-to-end pipeline (raw data → cleaned → features → model)
8. Mock SQL + pandas interview rounds (Pramp, interviewing.io)
9. Move on to the modeling stage — see the ML algorithms roadmap for the next step

This plan builds the exact practical skill set MAANG interviewers probe for in data science and ML engineering screens.